Taller

Endpoints en Django nativo (sin librerías externas)

Docente: Jose Javier Torres Vanstrahlen
Curso: Desarrollo de aplicaciones Web I — 4E

Repositorio:
https://github.com/5t6nfjn9pn-cmyk/tallerinvesticion

Usuario de GitHub: 5t6nfjn9pn-cmyk

1. Introducción

Este documento investiga cómo crear endpoints en Django nativo, sin librerías externas (como Django REST Framework), manejando los cuatro verbos HTTP principales: GET, POST, PUT y DELETE. Se utiliza una vista basada en clase cuyos métodos son **get()**, **post()**, **put()** y **delete()**. La clase `View` de Django decide qué método ejecutar según el verbo de la petición (mediante `dispatch()`). Si el verbo no está definido, responde 405 Method Not Allowed.

2. Verbos HTTP y operaciones CRUD

Verbo	CRUD	Caso de uso	Método	Códigos típicos
GET	Read	Listar todos los registros o consultar uno por pk	get()	200, 404
POST	Create	Registrar un nuevo recurso	post()	201, 400
PUT	Update	Reemplazar o modificar un recurso existente	put()	200, 400, 404
DELETE	Delete	Eliminar un recurso existente	delete()	204, 404

3. Módulos, funciones y decoradores usados

Módulos e imports

Elemento	Descripción
django.views.View	Clase base de las vistas basadas en clase. Redirige cada petición al método get(), post(), put() o delete() según el verbo.
django.http.JsonResponse	Devuelve datos como JSON con el Content-Type correcto. Con safe=False permite devolver listas.
django.http.HttpResponse	Respuesta HTTP genérica; se usa, por ejemplo, para un 204 sin cuerpo.
django.shortcuts.get_object_or_404	Busca un objeto y lanza Http404 si no existe.
django.forms.models.model_to_dict	Convierte una instancia de modelo en diccionario para serializarla a JSON.
json (librería estándar de Python)	json.loads(request.body) convierte el cuerpo de la petición en un diccionario.
django.urls.path	Define rutas en urls.py y captura parámetros como <int:pk>.
django.urls.include	Incluye el urls.py de la app dentro del urls.py del proyecto.
django.db.models.Model	Clase base del modelo; da acceso al ORM (objects.all(), create(), save(), delete()).

Decoradores

Decorador	Descripción
@csrf_exempt (django.views.decorators.csrf)	Desactiva la verificación del token CSRF. Necesario para probar POST, PUT y DELETE desde Postman o curl.
@method_decorator(csrf_exempt, name='dispatch') (django.utils.decorators)	Aplica un decorador de funciones a una clase. Se coloca sobre dispatch, que recibe todas las peticiones.

Objetos clave del request

Atributo	Descripción
request.method	Verbo HTTP de la petición.
request.body	Cuerpo en bytes. En PUT y DELETE no se llena request.POST, por eso se usa json.loads(request.body).
request.GET	Parámetros de la URL (?campo=valor).

4. Código del endpoint

Archivos que se modifican: **views.py** y **urls.py** (se crea si no existe), además de **models.py** si aún no hay modelo.

models.py

```
from django.db import models

class Proyecto(models.Model):
    nombre = models.CharField(max_length=100)
    descripcion = models.TextField(blank=True)

    def __str__(self):
        return self.nombre
```

views.py

```
# LIBRERIAS
import json
from django.views import View
from django.http import JsonResponse, HttpResponse
from django.shortcuts import get_object_or_404
from django.utils.decorators import method_decorator
from django.views.decorators.csrf import csrf_exempt
from django.forms.models import model_to_dict
from .models import Proyecto


# VISTA
@method_decorator(csrf_exempt, name='dispatch')
class ProyectoView(View):

    # READ: lista todos o devuelve uno si llega pk
    def get(self, request, pk=None):
        if pk is not None:
            proyecto = get_object_or_404(Proyecto, pk=pk)
            return JsonResponse(model_to_dict(proyecto))
        proyectos = list(Proyecto.objects.values())
        return JsonResponse(proyectos, safe=False)

    # CREATE
    def post(self, request):
        try:
            data = json.loads(request.body)
        except json.JSONDecodeError:
            return JsonResponse({'error': 'JSON invalido'}, status=400)

        if not data.get('nombre'):
            return JsonResponse({'error': 'El campo nombre es obligatorio'}, status=400)

        proyecto = Proyecto.objects.create(
            nombre=data['nombre'],
            descripcion=data.get('descripcion', '')
        )
        return JsonResponse(model_to_dict(proyecto), status=201)

    # UPDATE
    def put(self, request, pk):
        proyecto = get_object_or_404(Proyecto, pk=pk)
        try:
            data = json.loads(request.body)
        except json.JSONDecodeError:
            return JsonResponse({'error': 'JSON invalido'}, status=400)

        proyecto.nombre = data.get('nombre', proyecto.nombre)
        proyecto.descripcion = data.get('descripcion', proyecto.descripcion)
        proyecto.save()
        return JsonResponse(model_to_dict(proyecto))

    # DELETE
    def delete(self, request, pk):
        proyecto = get_object_or_404(Proyecto, pk=pk)
        proyecto.delete()
        return HttpResponse(status=204)
```

urls.py (de la app)

```
from django.urls import path
from .views import ProyectoView

urlpatterns = [
    path('proyectos/', ProyectoView.as_view()),           # GET (lista), POST
    path('proyectos/<int:pk>/', ProyectoView.as_view()),  # GET (uno), PUT, DELETE
]
```

urls.py (del proyecto)

```
from django.urls import path, include

urlpatterns = [
    path('api/', include('mi_app.urls')),
]
```

Migraciones

```
python manage.py makemigrations
python manage.py migrate
python manage.py runserver
```

5. Pruebas con curl

```
# POST
curl -X POST http://localhost:8000/api/proyectos/ -H "Content-Type: application/json" \
     -d '{"nombre":"Demo","descripcion":"Primero"}'
# GET
curl http://localhost:8000/api/proyectos/
# PUT
curl -X PUT http://localhost:8000/api/proyectos/1/ -H "Content-Type: application/json" \
     -d '{"nombre":"Demo editado"}'
# DELETE
curl -X DELETE http://localhost:8000/api/proyectos/1/
```

6. Fuentes consultadas

- Django Docs, Class-based views: Introduction: https://docs.djangoproject.com/en/stable/topics/class-based-views/intro/
- Django Docs, Built-in class-based views API (View): https://docs.djangoproject.com/en/stable/ref/class-based-views/base/
- Django Docs, Request and response objects: https://docs.djangoproject.com/en/stable/ref/request-response/
- Django Docs, URL dispatcher: https://docs.djangoproject.com/en/stable/topics/http/urls/
- Django Docs, Cross Site Request Forgery protection: https://docs.djangoproject.com/en/stable/howto/csrf/
- Django Docs, Django shortcut functions: https://docs.djangoproject.com/en/stable/topics/http/shortcuts/
- MDN Web Docs, HTTP request methods: https://developer.mozilla.org/es/docs/Web/HTTP/Methods

7. Notas

- **@csrf_exempt** es solo para desarrollo y pruebas. En producción se debe manejar CSRF o autenticación.
- Todas las respuestas usan JSON y códigos HTTP acordes a cada operación (200, 201, 204, 400, 404).